

Intégration de Modèles IA

dans les Systèmes Logiciels

Lab 5 : Intégrer un LLM via Groq

Dr. El Hadji Bassirou TOURE
École Supérieure Polytechnique
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

L2 GLSI / DUT-INFO2 — Semestre 4 — 2026

Résumé

Ce lab ajoute de l'**IA générative** à SénSanté. Vous créez un compte Groq, obtenez une clé API gratuite, écrivez un script de test, puis ajoutez un endpoint `POST /explain` dans l'API FastAPI qui appelle Llama 3 pour expliquer les diagnostics en français simple. Le frontend sera modifié pour afficher cette explication sous le résultat. Tag final : `v4`.

🕒 Ce que vous allez apprendre

À l'issue de ce lab, vous saurez :

- Créer un compte Groq et obtenir une clé API
- Stocker une clé API de manière sécurisée (fichier `.env`)
- Appeler un LLM (Llama 3) via l'API Groq en Python
- Écrire un system prompt adapté à un contexte médical
- Ajouter un endpoint `POST /explain` dans FastAPI
- Connecter le frontend pour afficher l'explication du LLM

Durée estimée : 2 heures

Prérequis : Lab 4 terminé (SénSanté avec formulaire + diagnostic fonctionnel, tag `v3`)

Tag Git : `v4` | **Compétence :** C6 (Intégrer un LLM via API)

Table des matières

1	Introduction : le modèle qui parle	2
2	Étape 1 : Créer un compte Groq et obtenir la clé API	2
3	Étape 2 : Stocker la clé de manière sécurisée	3
4	Étape 3 : Premier appel Groq — script de test	4
5	Étape 4 : Ajouter l'endpoint <code>POST /explain</code> dans l'API	6
6	Étape 5 : Tester l'endpoint <code>/explain</code>	8

7	Étape 6 : Connecter le frontend	9
8	Étape 7 : Tester l'application complète	10
9	Étape 8 : Git — sauvegarder la version v4	11
10	Résumé	13
11	Exercices	13
12	Conclusion	14

1 Introduction : le modèle qui parle

SénSanté fonctionne. Awa, agente de santé à Ziguinchor, remplit le formulaire, clique sur « Diagnostiquer », et voit : **paludisme — 72%**. C'est utile, mais Awa demande : « Pourquoi paludisme et pas grippe ? Et je fais quoi maintenant ? »

Le modèle RandomForest ne peut pas répondre. Il classe, mais ne parle pas. Pour expliquer, il faut un modèle qui comprend le langage : un **LLM**.

En CM4, on a vu que Groq offre une API gratuite et ultra-rapide pour Llama 3. Ce lab l'intègre dans SénSanté.

Flux de l'endpoint POST /explain

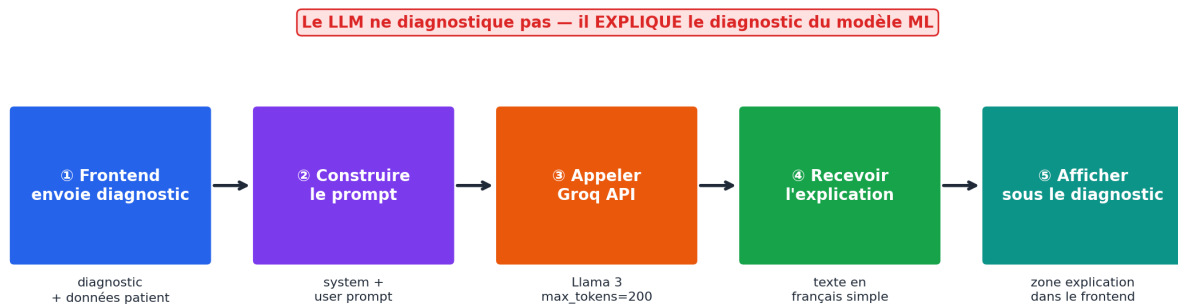


FIGURE 1 – Le flux de l'endpoint POST /explain. Le diagnostic du modèle ML est envoyé au LLM qui génère une explication en français.

★ Intuition

Analogie : le traducteur

Le modèle ML est un **laboratoire d'analyses** : il reçoit les symptômes et produit un résultat technique ("paludisme, 72%"). Le LLM est le **médecin** qui traduit ce résultat en langage humain : "Vos symptômes sont cohérents avec un paludisme. Consultez un centre de santé pour un test rapide."

Les deux font un travail différent. Ensemble, ils offrent une expérience complète.

2 Étape 1 : Créer un compte Groq et obtenir la clé API

☒ À faire

Étape 1.1 : Créer un compte Groq

1. Allez sur <https://console.groq.com>
2. Cliquez sur **Sign Up** (inscription gratuite, pas de carte bancaire)
3. Connectez-vous avec Google ou créez un compte email

☒ À faire

Étape 1.2 : Générer une clé API

1. Dans la console Groq, allez dans **API Keys**
2. Cliquez sur **Create API Key**
3. Nommez-la **sensante**
4. **Copiez la clé immédiatement** — elle ne sera plus affichée ensuite
La clé ressemble à : `gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXX`

△ Point d'attention

Ne partagez JAMAIS cette clé. Ne la mettez pas dans votre code Python. Ne la committez pas dans Git. La section suivante explique comment la stocker de manière sécurisée.

3 Étape 2 : Stocker la clé de manière sécurisée

Sécurité de la clé API : le fichier .env

MAUVAISE pratique

```
client = Groq(
    api_key="gsk_abc123..."
)
```

→ Clé visible dans le code
→ Publiée sur GitHub
→ Volée par n'importe qui

BONNE pratique

```
# .env (dans .gitignore)
GROQ_API_KEY=gsk_abc123...

# main.py
import os
api_key=os.getenv("GROQ_API_KEY")

→ Clé jamais dans Git
```

FIGURE 2 – La bonne pratique : stocker la clé API dans un fichier `.env` qui est dans le `.gitignore`. Ne jamais écrire la clé dans le code.

☒ À faire

Étape 2.1 : Installer `python-dotenv`

```
pip install groq python-dotenv
pip freeze > requirements.txt
```

☒ À faire

Étape 2.2 : Créer le fichier `.env` à la racine du projet

```
# .env (à la racine de sensante/)
GROQ_API_KEY=gsk_votre_cle_ici
```

☒ À faire**Étape 2.3 : Ajouter .env au .gitignore**

Ouvrez .gitignore et ajoutez :

```
# Cles API - NE JAMAIS COMMITTER
.env
```

• Point clé à retenir

Vérification cruciale : Tapez `git status`. Le fichier `.env` ne doit **pas** apparaître dans la liste des fichiers à ajouter. Si vous le voyez, c'est que le `.gitignore` n'est pas correct.

4 Étape 3 : Premier appel Groq — script de test

☒ À faire**Étape 3.1 : Créer notebooks/test_groq.py**

```
# notebooks/test_groq.py
# Test de l'API Groq avec Llama 3

import os
from dotenv import load_dotenv
from groq import Groq

# Charger la cle depuis .env
load_dotenv()
api_key = os.getenv("GROQ_API_KEY")

if not api_key:
    print("ERREUR : GROQ_API_KEY non trouvee dans .env")
    exit()

# Creer le client Groq
client = Groq(api_key=api_key)

# Premier appel : question simple
response = client.chat.completions.create(
    model="llama-3.1-8b-instant",
    messages=[
        {"role": "system",
         "content": "Tu es un assistant medical senegalais. "
                    "Reponds en francais simple. "
                    "Maximum 3 phrases."},
        {"role": "user",
         "content": "Quels sont les symptomes du paludisme ?"}
    ],
    max_tokens=200,
    temperature=0.3
)

# Afficher la reponse
```

```
print("=== Reponse de Llama 3 ===")
print(response.choices[0].message.content)
print(f"\nTokens utilises : {response.usage.total_tokens}")
```

☒ À faire

Étape 3.2 : Exécuter le test

```
python notebooks/test_groq.py
```

```
=== Reponse de Llama 3 ===
Les symptomes du paludisme incluent une forte fièvre,
des maux de tete, des frissons et une fatigue intense.
En saison des pluies au Senegal, ces symptomes doivent
etre pris au serieux. Consultez un centre de sante pour
un test de diagnostic rapide.
```

```
Tokens utilises : 87
```

• Point clé à retenir

Ça fonctionne ! En 20 lignes, vous avez appelé un modèle de 8 milliards de paramètres qui a répondu en français en moins d'une seconde. C'est la puissance de l'API LLM.

☒ À faire

Étape 3.3 : Tester avec un diagnostic SènSanté

```
# Test avec le format SenSante
response2 = client.chat.completions.create(
    model="llama-3.1-8b-instant",
    messages=[
        {"role": "system",
         "content": """Tu es un assistant medical senegalais.
Tu recois un diagnostic et des donnees patient.
Explique le resultat en francais simple,
comme un medecin parlerait a son patient.
Sois rassurant mais recommande une consultation.
Maximum 3 phrases.
Ne fais JAMAIS de diagnostic toi-meme."""},
        {"role": "user",
         "content": """Patient : Femme, 28 ans, region Dakar
Symptomes : temperature 39.5, toux, fatigue, maux de tete
Diagnostic du modele : paludisme (probabilite 72%)
Explique ce resultat au patient."""}
    ],
    max_tokens=200,
    temperature=0.3
)

print("=== Explication SenSante ===")
print(response2.choices[0].message.content)
```

```
=== Explication SenSante ===
Vos symptomes de fièvre elevee, toux, fatigue et maux
de tete, observes en periode de pluies a Dakar, sont
coherents avec une suspicion de paludisme. Le modele
indique une probabilite de 72%, ce qui est significatif.
Je vous recommande de consulter rapidement un centre
de sante pour un test de diagnostic rapide (TDR).
```

5 Étape 4 : Ajouter l'endpoint POST /explain dans l'API

☒ À faire

Étape 4.1 : Ajouter les imports et le client Groq dans api/main.py

Ajoutez au début du fichier, après les imports existants :

```
import os
from dotenv import load_dotenv
from groq import Groq

# Charger les variables d'environnement
load_dotenv()

# Client Groq (charge au démarrage)
groq_client = None
groq_api_key = os.getenv("GROQ_API_KEY")
if groq_api_key:
    groq_client = Groq(api_key=groq_api_key)
    print("Client Groq initialise.")
else:
    print("ATTENTION : GROQ_API_KEY non trouvee. "
          "/explain sera desactive.")
```

☒ À faire

Étape 4.2 : Ajouter le schéma Pydantic pour /explain

```
from pydantic import BaseModel, Field

class ExplainInput(BaseModel):
    diagnostic: str = Field(...,
        description="Diagnostic predit par le modele")
    probabilite: float = Field(...,
        description="Probabilite du diagnostic")
    age: int = Field(...)
    sexe: str = Field(...)
    temperature: float = Field(...)
    region: str = Field(...)

class ExplainOutput(BaseModel):
```

```

explication: str = Field(...,
    description="Explication en francais")
modele_llm: str = Field(
    default="llama-3.1-8b-instant",
    description="Modele LLM utilise")

```

☒ À faire

Étape 4.3 : Ajouter la route POST /explain

```

SYSTEM_PROMPT = """Tu es un assistant medical senegalais.
Tu recois un diagnostic et des donnees patient.
Explique le resultat en francais simple,
comme un medecin parlerait a son patient.
Sois rassurant mais recommande toujours
une consultation medicale.
Maximum 3 phrases.
Ne fais JAMAIS de diagnostic toi-meme.
Tu expliques uniquement le diagnostic fourni."""

@app.post("/explain", response_model=ExplainOutput)
def explain(data: ExplainInput):
    """Expliquer un diagnostic en francais avec un LLM."""
    if not groq_client:
        return ExplainOutput(
            explication="Service d'explication indisponible. "
            "Cle API non configuree.",
            modele_llm="aucun"
        )

    # Construire le user prompt
    user_prompt = (
        f"Patient : {data.sexe}, {data.age} ans, "
        f"region {data.region}\n"
        f"Temperature : {data.temperature} C\n"
        f"Diagnostic du modele : {data.diagnostic} "
        f"(probabilite {data.probabilite:.0%})\n"
        f"Explique ce resultat au patient."
    )

    try:
        response = groq_client.chat.completions.create(
            model="llama-3.1-8b-instant",
            messages=[
                {"role": "system",
                 "content": SYSTEM_PROMPT},
                {"role": "user",
                 "content": user_prompt}
            ],
            max_tokens=200,
            temperature=0.3
        )
        explication = response.choices[0].message.content
    except Exception as e:
        explication = (
            f"Erreur lors de l'appel au LLM : {str(e)}")

```



```
return ExplainOutput(explication=explication)
```

□ Explication pédagogique

Points importants dans ce code :

- **Graceful degradation** : si la clé API n'est pas configurée, `/explain` renvoie un message d'erreur propre au lieu de planter. L'app fonctionne sans le LLM.
- **User prompt dynamique** : il est construit à partir des données réelles du patient et du diagnostic du modèle ML.
- **try/except** : si Groq est injoignable (réseau, quota), on gère l'erreur proprement.
- **temperature=0.3** : réponses déterministes et factuelles (pas créatives — c'est médical).

6 Étape 5 : Tester l'endpoint `/explain`

☒ À faire

Étape 5.1 : Relancer l'API

```
uvicorn api.main:app --reload
```

```
Chargement du modele...
Modele charge : ['grippe', 'palu', 'sain', 'typh']
Client Groq initialise.
INFO:      Application startup complete.
```

☒ À faire

Étape 5.2 : Tester via Swagger UI

Allez sur <http://localhost:8000/docs>, cliquez sur **POST** `/explain`, puis « Try it out » avec :

```
{
  "diagnostic": "palu",
  "probabilite": 0.72,
  "age": 28,
  "sexe": "F",
  "temperature": 39.5,
  "region": "Dakar"
}
```

Réponse attendue :

```
{
  "explication": "Vos symptomes de fièvre elevee a
    39.5 C, observes a Dakar, sont coherents avec
    une suspicion de paludisme...",
  "modele_llm": "llama-3.1-8b-instant"
}
```

• Point clé à retenir

Le LLM est intégré dans l'API. Vous avez maintenant 3 endpoints : /health, /predict (ML), et /explain (LLM). SènSanté combine les deux types d'IA.

7 Étape 6 : Connecter le frontend

☒ À faire

Étape 6.1 : Modifier la fonction `afficherResultat()` dans `frontend/index.html`

Après l'affichage du diagnostic, ajoutez l'appel automatique à `/explain` :

```
function afficherResultat(r) {
  // ... (code existant du Lab 4 pour
  //   afficher diagnostic + barre) ...

  // NOUVEAU : appeler /explain automatiquement
  demanderExplication(r);
}

async function demanderExplication(diagnostic_data) {
  // Recuperer les donnees du formulaire
  const explainData = {
    diagnostic: diagnostic_data.diagnostic,
    probabilite: diagnostic_data.probabilité,
    age: parseInt(
      document.getElementById("age").value),
    sexe: document.getElementById("sexe").value,
    temperature: parseFloat(
      document.getElementById("temperature")
        .value),
    region:
      document.getElementById("region").value
  };

  // Zone d'explication dans le result-section
  const section =
    document.getElementById("result-section");
  // Ajouter un placeholder
  const explDiv = document.createElement("div");
  explDiv.id = "explication-zone";
  explDiv.innerHTML = `
    <div class="mt-4 bg-purple-50 rounded-lg p-4
      border border-purple-200">
      <p class="text-sm text-purple-600
        font-medium mb-2">
        Explication (Llama 3)</p>
      <p class="text-gray-500 text-sm
        animate-pulse">
        Generation en cours...</p>
    </div>`;
  section.querySelector(
    ".bg-white").appendChild(explDiv);

  try {
```

```

const response = await fetch(
  API_URL + "/explain", {
    method: "POST",
    headers:
      {"Content-Type": "application/json"},
    body: JSON.stringify(explainData)
  });

if (response.ok) {
  const result = await response.json();
  explDiv.innerHTML = `
    <div class="mt-4 bg-purple-50
      rounded-lg p-4 border
      border-purple-200">
      <p class="text-sm text-purple-600
        font-medium mb-2">
        Explication (Llama 3)</p>
      <p class="text-gray-700 text-sm
        leading-relaxed">
        ${result.explication}</p>
    </div>`;
}
} catch (error) {
  explDiv.innerHTML = `
    <div class="mt-4 bg-gray-50 rounded-lg
      p-3">
      <p class="text-gray-400 text-sm">
        Explication non disponible.</p>
    </div>`;
}
}

```

□ Explication pédagogique

Le flux complet en action :

1. Awa clique sur « Diagnostiquer »
 2. `fetch()` appelle `POST /predict` → diagnostic + probabilité
 3. Le diagnostic s'affiche immédiatement
 4. En parallèle, `fetch()` appelle `POST /explain` → Groq → explication
 5. L'explication s'affiche sous le diagnostic (avec "Génération en cours..." pendant l'attente)
- L'utilisateur voit le diagnostic **instantanément** et l'explication arrive 1–2 secondes plus tard. C'est un bon pattern UX : ne pas bloquer l'affichage en attendant le LLM.

8 Étape 7 : Tester l'application complète

☒ À faire

Étape 7.1 : Tester bout en bout

1. Vérifiez que l'API tourne (`uvicorn api.main:app -reload`)
2. Ouvrez le frontend (`http://localhost:3000` ou double-clic sur `index.html`)
3. Remplissez : Âge = 28, F, 39.5°C, 110, Toux + Fatigue + Maux de tête, Dakar
4. Cliquez sur **Diagnostiquer**

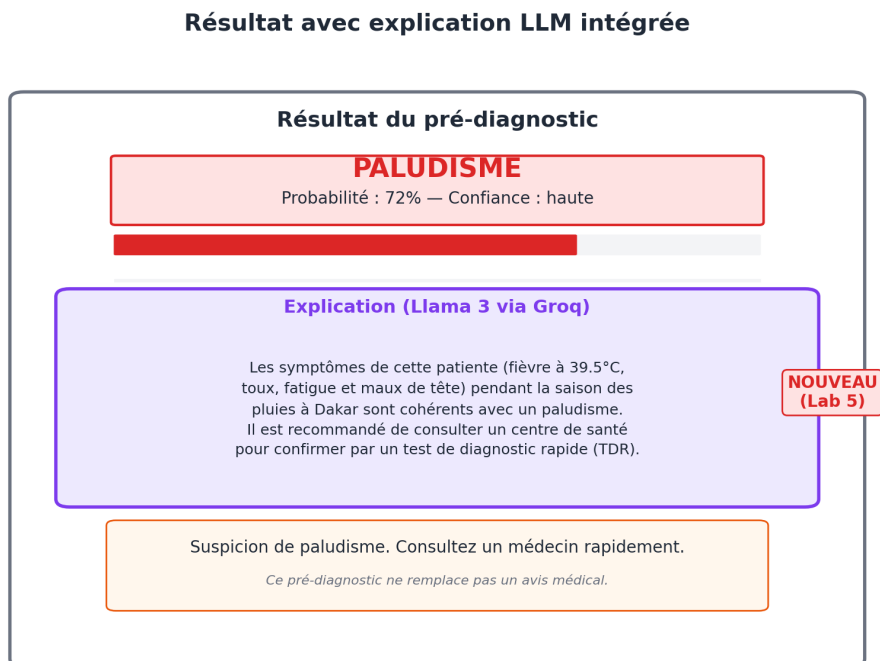


FIGURE 3 – Le résultat avec l’explication LLM intégrée : diagnostic en haut, barre de probabilité, puis explication en français générée par Llama 3.

5. Vous devriez voir le diagnostic **puis** l’explication en français

☒ À faire

Étape 7.2 : Tester les cas limites

- Patient sain (pas de symptômes, 37°C) → l’explication doit être rassurante
- Diagnostic avec confiance faible → l’explication doit le mentionner
- Supprimer la clé de `.env` et relancer → diagnostic fonctionne, explication dégradée gracieusement

• Point clé à retenir

ML classique + LLM dans la même application.

C’est l’architecture complète de SénSanté. Le RandomForest diagnostique. Llama 3 explique. Le frontend affiche les deux. L’API orchestre le tout. C’est exactement la vision du CM4.

9 Étape 8 : Git — sauvegarder la version v4

☒ À faire

Étape 8.1 : Vérifier que `.env` est ignoré

```
git status
```

Vérifiez que `.env` n’apparaît pas dans la liste. Si c’est le cas, votre `.gitignore` fonctionne.

☒ À faire**Étape 8.2 : Commit et tag**

```
git add .
git commit -m "Lab 5 : integration LLM Groq + /explain (v4)"
git push origin main
git tag v4
git push origin v4
```

```
[main f6a7b8c] Lab 5 : integration LLM Groq + /explain (v4)
4 files changed, 97 insertions(+), 3 deletions(-)
create mode 100644 notebooks/test_groq.py
To https://github.com/moussa-diallo/sensante.git
* [new tag]          v4 -> v4
```

• Point clé à retenir**Historique Git après 5 labs :**

```
git log --oneline
f6a7b8c Lab 5 : integration LLM Groq (v4)
e5f6a7b Lab 4 : frontend Tailwind CSS (v3)
d4e5f6a Lab 3 : API FastAPI (v2)
a3b7c21 Lab 2 : modele RandomForest (v1)
f1e2d3c Lab 1 : structure projet (v0)
```

Cinq incréments, cinq tags. Il ne reste plus qu'un lab : Docker et déploiement.

10 Résumé

Structure du projet après Lab 5

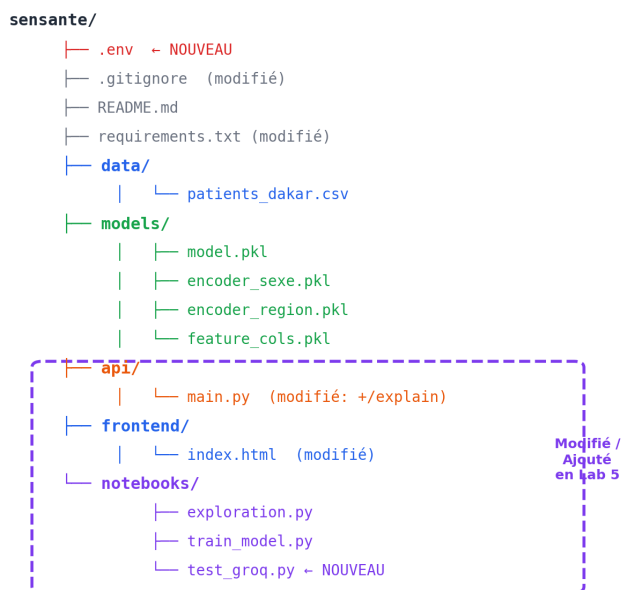


FIGURE 4 – Structure du projet après Lab 5. Nouveaux fichiers : `.env`, `test_groq.py`. Fichiers modifiés : `api/main.py`, `frontend/index.html`, `.gitignore`, `requirements.txt`.

• Point clé à retenir

Ce que vous avez accompli dans ce lab :

#	Action	Statut
1	Créer un compte Groq + clé API gratuite	✓
2	Stocker la clé dans <code>.env</code> + <code>.gitignore</code>	✓
3	Script de test Groq (<code>test_groq.py</code>)	✓
4	System prompt médical SènSanté	✓
5	Endpoint POST <code>/explain</code> dans FastAPI	✓
6	Frontend : afficher l'explication sous le diagnostic	✓
7	Tests bout en bout + cas limites	✓
8	Git : <code>commit</code> + <code>push</code> + <code>tag v4</code>	✓

11 Exercices

☒ À faire

Exercice 1 — Prompt engineering : Modifiez le system prompt pour que le LLM réponde en **wolof simple** (ou en français mélangé de termes wolof). Testez avec plusieurs diagnostics. Le LLM arrive-t-il à mélanger correctement ?

Exercice 2 — Temperature : Testez le même appel avec `temperature=0.0`, `0.5`, et `1.0`.

Décrivez en 3 phrases comment la réponse change.

Exercice 3 — Réflexion éthique : Un patient lit l'explication du LLM et décide de se soigner seul sans consulter un médecin. Est-ce un risque ? Proposez 2 modifications de l'interface pour minimiser ce risque.

12 Conclusion

☉ Ce que vous allez apprendre

Félicitations ! Vous avez :

- ✓ Intégré un LLM (Llama 3) dans une application Python
- ✓ Appris à sécuriser une clé API avec `.env`
- ✓ Ajouté l'endpoint `POST /explain` dans FastAPI
- ✓ Combiné ML classique et IA générative dans la même app
- ✓ Créé le tag `v4` sur GitHub

SénSanté est maintenant **fonctionnellement complet** : formulaire → modèle ML → diagnostic → LLM → explication. Mais l'application ne tourne que sur *votre* machine. Awa à Ziguinchor ne peut pas y accéder.

En Lab 6 : Vous empaqueteriez SénSanté dans **Docker** (un conteneur portable) et le déploieriez sur **Hugging Face Spaces** — une URL publique accessible à n'importe qui dans le monde. Le tag passera à `v1.0` : la première version complète et déployée. Le deployment gap sera officiellement comblé.

Références

- [1] C. Huyen, *AI Engineering*, O'Reilly, 2025. Chapitres 1–3.
- [2] Documentation Groq : <https://console.groq.com/docs>
- [3] Documentation python-dotenv : <https://pypi.org/project/python-dotenv/>
- [4] Meta AI, “Llama 3 Model Card”, 2024. <https://ai.meta.com/llama/>